

Trabajo Práctico 7: Comparación de Threads y Async Programming

Programación Concurrente - Primer cuatrimestre 2025

Profesores: Emilio Lopez Gabeiras y Rodrigo Pazos

10 de junio de 2025

1. Introducción

Este trabajo práctico busca comparar dos enfoques de programación concurrente: el modelo tradicional basado en *threads* y el modelo basado en *async programming*. Se implementarán tareas tanto de simulación de I/O como de cómputo intensivo, utilizando ambos enfoques. El objetivo es observar y analizar diferencias. **Es necesario utilizar la librería Tokio de Rust.**

2. Objetivos

1. Implementar un sistema concurrente para simular tareas de I/O y cálculo intensivo, usando tanto threads como async (`task::spawn` en Tokio).
2. Medir y comparar tiempos de ejecución, uso de memoria y comportamiento de CPU en ambos modelos.
3. Analizar diferencias de performance entre el enfoque orientado a I/O y al cálculo numérico.

3. Implementación

El código debe incluir las siguientes funcionalidades clave, implementadas utilizando tanto el enfoque de `threads` como el de `async/await`:

- **Simulación de tareas I/O:** Cada tarea debe simular una operación de I/O con una espera artificial. En la versión con `threads`, esto se puede lograr usando `std::thread::sleep`. En la versión `async`, debe utilizarse `tokio::time::sleep` para evitar bloquear el thread principal.
- **Cálculo concurrente de Pi con la serie de Leibniz:** Se debe dividir el trabajo del cálculo en subtareas que se ejecuten en paralelo o de forma asincrónica. Cada subtarea debe calcular una parte de la serie (ver función `leibniz_pi_partial` en la sección 4), y al finalizar se deben combinar los resultados parciales sumándolos para obtener la aproximación final del valor de π .
 - En la versión con `threads`, se pueden lanzar varios threads y usar `join` para recolectar los resultados.
 - En la versión `async`, se pueden usar `tokio::spawn` o `futures::join` para ejecutar múltiples tareas concurrentes.
 - El número de tareas y la cantidad de términos a calcular deben ser configurables mediante argumentos o constantes.

4. Instrucciones para la experimentación

Ejecutar el programa probando distintas combinaciones de parámetros para observar cómo varían los tiempos y el comportamiento de CPU en función del número de tareas y la cantidad de términos para el cálculo de Pi.

- **tasks:** número de tareas concurrentes que se lanzan (por ejemplo, 10, 100, 1000, 1 000 000).
- **terms:** cantidad total de términos usados para la serie de Leibniz en el cálculo de Pi (por ejemplo, 10 000, 1 000 000, 10 000 000). Estos términos se dividen equitativamente entre las tareas.

Para el cálculo de Pi utilizando la serie de Leibniz, se puede usar la siguiente función auxiliar:

```
1 fn leibniz_pi_partial(start: usize, count: usize) -> f64 {  
2     (start..start + count)  
3     .map(|k| {  
4         let sign = if k % 2 == 0 { 1.0 } else { -1.0 };  
5         sign / (2.0 * k as f64 + 1.0)  
6     })  
7     .sum::<f64>()  
8 }
```

Listing 1: Función para cálculo parcial de la serie de Leibniz

Esta función calcula una porción de la serie de Leibniz sin el factor final. Cada tarea calcula su porción parcial, y al finalizar se deben **sumar todos los resultados parciales y multiplicar por 4** para obtener la aproximación de π :

```
1 let pi: f64 = partial_results.iter().sum::<f64>() * 4.0;
```

Listing 2: Combinación de resultados parciales

4.1. Métricas a capturar

Para cada ejecución, registrar:

- **Tiempo wall-clock:** usar `std::time::Instant` al inicio y fin de la ejecución.

- **Éxito o fallo:** registrar si la ejecución completó correctamente o falló (por ejemplo, por límite de threads del sistema operativo).

Se recomienda ejecutar cada combinación al menos 3 veces y reportar el promedio. Los resultados deben exportarse a un archivo CSV con el siguiente formato, para poder cargarlo en Google Sheets y generar gráficos comparativos:

```
1 modelo , tipo , tasks , terms , run , time_ms , status
2 threads , io , 1000 , , 1 , 1023 , ok
3 async , io , 1000 , , 1 , 15 , ok
4 threads , cpu , 8 , 10000000 , 1 , 520 , ok
5 async , cpu , 8 , 10000000 , 1 , 535 , ok
```

Listing 3: Formato CSV de resultados

5. Análisis sugerido

- Compare los tiempos de ejecución para I/O simulado entre threads y async con diferentes valores de tasks. ¿Cuál es más eficiente en manejar muchas tareas con esperas? Tip: probar con valores altos (100K+) y explicar por qué threads falla y async no. ¿Qué recurso del sistema se agota?
- Compare los tiempos de ejecución para cálculo de Pi intensivo en CPU entre threads y async. ¿Qué modelo se desempeña mejor? ¿Son muy diferentes los resultados? ¿Por qué?
- Experimente con diferentes cantidades de tareas y términos para Pi para evaluar cómo escala cada enfoque. ¿En qué punto el overhead de crear threads supera el beneficio del paralelismo?
- **Reflexión:** Dado que `tokio::spawn` ejecuta tareas en un thread pool, ¿qué diferencia real hay entre threads y async para tareas CPU-bound que no hacen `.await`? ¿Cuándo convendría usar `tokio::task::spawn_blocking`?